Absolute Return Forecasting: From Linear Models to LSTM

Edouard Blanc Clément Dardàre
Julien Simeoni

Université Paris Dauphine-PSL, Master 2 Machine Learning for Finance

March 29, 2026

Contents

1 Introduction **3**
1.1 Motivation . . . 3
1.2 Objective . . . 3
1.3 Dataset . . . 3
1.4 Evaluation . . . 4
1.5 Paper Structure . . . 4

2 Data **4**
2.1 Dataset Construction . . . 4
2.2 Descriptive Statistics . . . 5
2.3 Log-Return Time Series . . . 5
2.4 Realized Volatility and Regime Identification . . . 6
2.5 Autocorrelation Analysis . . . 6
2.6 Feature Correlation Structure . . . 7
2.7 Summary . . . 7

3 Linear Regression Baseline **8**
3.1 Standard Linear Regression . . . 8
3.1.1 Setup . . . 8
3.1.2 Results . . . 8
3.1.3 Coefficient Analysis . . . 8
3.1.4 Predicted vs. Actual . . . 9
3.2 Feature Engineering . . . 9
3.3 Regime Analysis . . . 10
3.4 GARCH(1,1) Baseline . . . 10
3.4.1 Estimated Parameters . . . 10
3.4.2 Regime Performance . . . 11
3.5 Summary of Limitations . . . 11

4 Feedforward Neural Network **11**
4.1 Architecture . . . 11
4.2 Training Setup . . . 12
4.3 Hyperparameter Study . . . 13
4.3.1 Activation Functions . . . 13
4.3.2 Weight Initialization . . . 14
4.3.3 Optimizer and Learning Rate . . . 14
4.4 Internal Network Analysis . . . 17
4.4.1 Activation and Weight Distributions . . . 17
4.4.2 Gradient Flow . . . 18
4.4.3 Weight Evolution During Training . . . 19
4.5 Lag Window Sensitivity . . . 19
4.6 Results . . . 20
4.7 Structural Limitations . . . 21
4.7.1 Order Blindness . . . 21
4.7.2 Regime Blindness . . . 23

5 Recurrent Neural Network **24**
5.1 Architecture . . . 24
5.2 Training Setup . . . 24
5.3 Results . . . 24
5.4 Limitations . . . 24

6 Long Short-Term Memory **24**
6.1 Architecture . . . 24
6.2 Training Setup . . . 24
6.3 Results . . . 24
6.4 Cross-Model Comparison . . . 24
6.5 Conclusion . . . 24

1 Introduction

1.1 Motivation

Accurately forecasting the magnitude of future asset returns is a central problem in quantitative finance. While the *direction* of price changes is notoriously difficult to predict, the *amplitude* of those changes (often proxied by realized volatility) exhibits strong and exploitable serial dependence. This phenomenon is characterised by *volatility clustering*: large price moves tend to be followed by large moves, and calm periods persist until a new shock arrives.

Predicting the absolute return of the S&P 500 index therefore serves a dual purpose: it is both a well-posed statistical problem with a meaningful signal in the data and a practically relevant task for risk management, option pricing, and portfolio construction.

1.2 Objective

This project empirically compares a progression of models of increasing expressive power applied to the same forecasting task:

1. **Linear Regression** — a transparent but structurally constrained baseline.
2. **GARCH(1,1)** — the industry-standard parametric volatility model.
3. **Feedforward Neural Network (FNN)** — a non-linear model that treats the input as a flat, orderless feature vector.
4. **Recurrent Neural Network (RNN)** — a sequential model that can exploit the temporal ordering of lags.
5. **Long Short-Term Memory (LSTM)** — a gated architecture designed to capture long-range dependencies.

Rather than searching for the best possible model, the goal is to *understand the limitations of each architecture* and to show how each successive model overcomes (some of) the shortcomings of its predecessor.

1.3 Dataset

The dataset is built from the daily log-returns of the SPY ETF (S&P 500 tracker), downloaded via the `yfinance` library. The sample spans **February 1993 to March 2026**, yielding **8 343 daily observations**.

The forecasting problem is formulated as a sliding-window regression:

- **Input** X_t: the L most recent daily log-returns $(r_{t-1}, r_{t-2}, \ldots, r_{t-L})$, with $L \geq 10$.
- **Target** y_t: the realized absolute return on day t, computed as the rolling standard deviation of returns over a short window (a proxy for next-day volatility).

1.4 Evaluation

Models are evaluated using three metrics, computed on a held-out test set:

- **MAE** (Mean Absolute Error) — primary metric for cross-model comparison.
- **RMSE** (Root Mean Squared Error) — penalises large errors more heavily.
- **QLIKE** (Quasi-Likelihood loss) — a standard volatility-forecasting loss that penalises systematic under-prediction.

Because market conditions vary dramatically, all models are also evaluated separately on two **market regimes** identified from the data:

- **Calm regime** (≈90% of observations): low, stable volatility.
- **Crisis regime** (≈10% of observations): sharp spikes corresponding to the dot-com crash, the 2008 Global Financial Crisis, and the COVID shock.

The cross-model summary (Section 6) reports MAE only.

1.5 Paper Structure

Section 2 describes the dataset and its key statistical properties. Section 3 presents the linear regression and GARCH baselines. Sections 4–6 progressively introduce the neural network architectures. A final comparison table closes the report.

2 Data

2.1 Dataset Construction

Daily log-returns for the SPY ETF are downloaded via `yfinance` and stored locally in `data/df_sp_500_log_ret.csv`. The raw return on day t is defined as

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right),$$

where P_t denotes the adjusted closing price. A sliding-window dataset is then constructed with a look-back of $L \geq 10$ lags:

$$\begin{aligned} X_t &= (r_{t-1},\, r_{t-2},\, \ldots,\, r_{t-L}) \in \mathbb{R}^L, \\ y_t &= \sigma_t \in \mathbb{R}_+, \end{aligned}$$

where σ_t is the realized absolute return (rolling standard deviation of returns over a short window), serving as a proxy for next-day volatility.

2.2 Descriptive Statistics

Table 1 reports summary statistics for the full sample.

Table 1: Descriptive statistics for S&P 500 daily log-returns (SPY, 1993–2026).

Statistic	Value
Observations	8 343
Start date	1993-02-01
End date	2026-03-24
Mean	0.0004
Std deviation	0.0117
Minimum	−0.1159
Maximum	+0.1356
Skewness	−0.248
Kurtosis	11.42

The distribution exhibits **negative skewness** (large negative shocks more common than large positive ones) and **excess kurtosis** of 11.42, confirming heavy tails relative to a Gaussian distribution. Both features are consistent with the well-known stylised facts of equity returns.

2.3 Log-Return Time Series

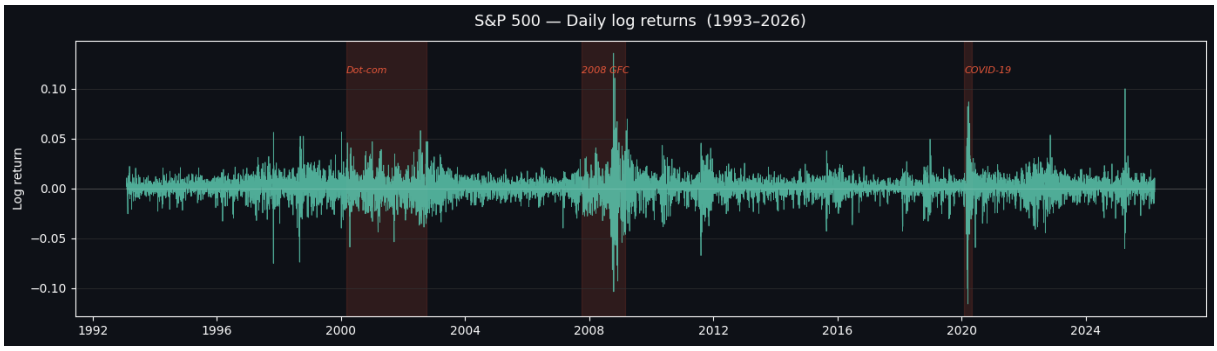

Figure 1: S&P 500 daily log-returns. Three major stress periods are clearly visible.

Returns oscillate around zero with no discernible trend, consistent with the efficient market hypothesis. Three major stress episodes are visible: the dot-com crash (2000–2002), the Global Financial Crisis (2008–2009), and the COVID shock (2020). These episodes exhibit the characteristic feature of **volatility clustering**: large moves arrive in bursts and calm periods are interspersed.

2.4 Realized Volatility and Regime Identification

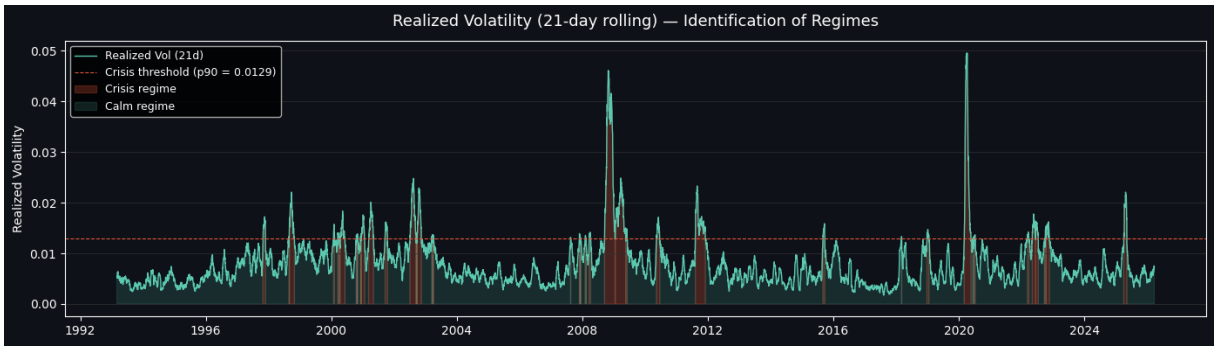

Figure 2: Realized absolute return (target variable y_t). The dashed line indicates the 90th-percentile threshold used to define the crisis regime.

A threshold at the **90th percentile** ($\sigma^* = 1.117$) is used to split the sample into two regimes:

Table 2: Market regime statistics.

Regime	Share of observations	Characterization
Calm	90%	Low, stable volatility
Crisis	10%	Sharp spikes; three major episodes

The pronounced asymmetry in regime frequency is critical: a model that performs well on average may still fail badly during the 10% of days that matter most for risk management.

2.5 Autocorrelation Analysis

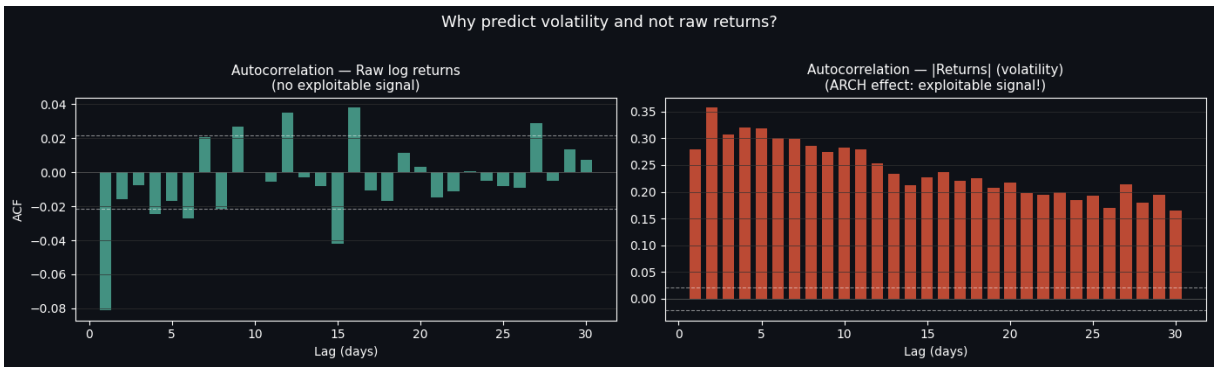

Figure 3: Autocorrelation of raw log-returns (near zero) vs absolute log-returns (strongly positive). The latter justifies using past absolute returns as features.

Two key findings:

- **Raw log-returns**: autocorrelation is essentially zero at all lags — the return series itself is unpredictable.

- **Absolute log-returns / realized volatility**: autocorrelation is strongly and persistently positive (the ARCH effect), justifying the use of past lagged returns as features for predicting future volatility.

2.6 Feature Correlation Structure

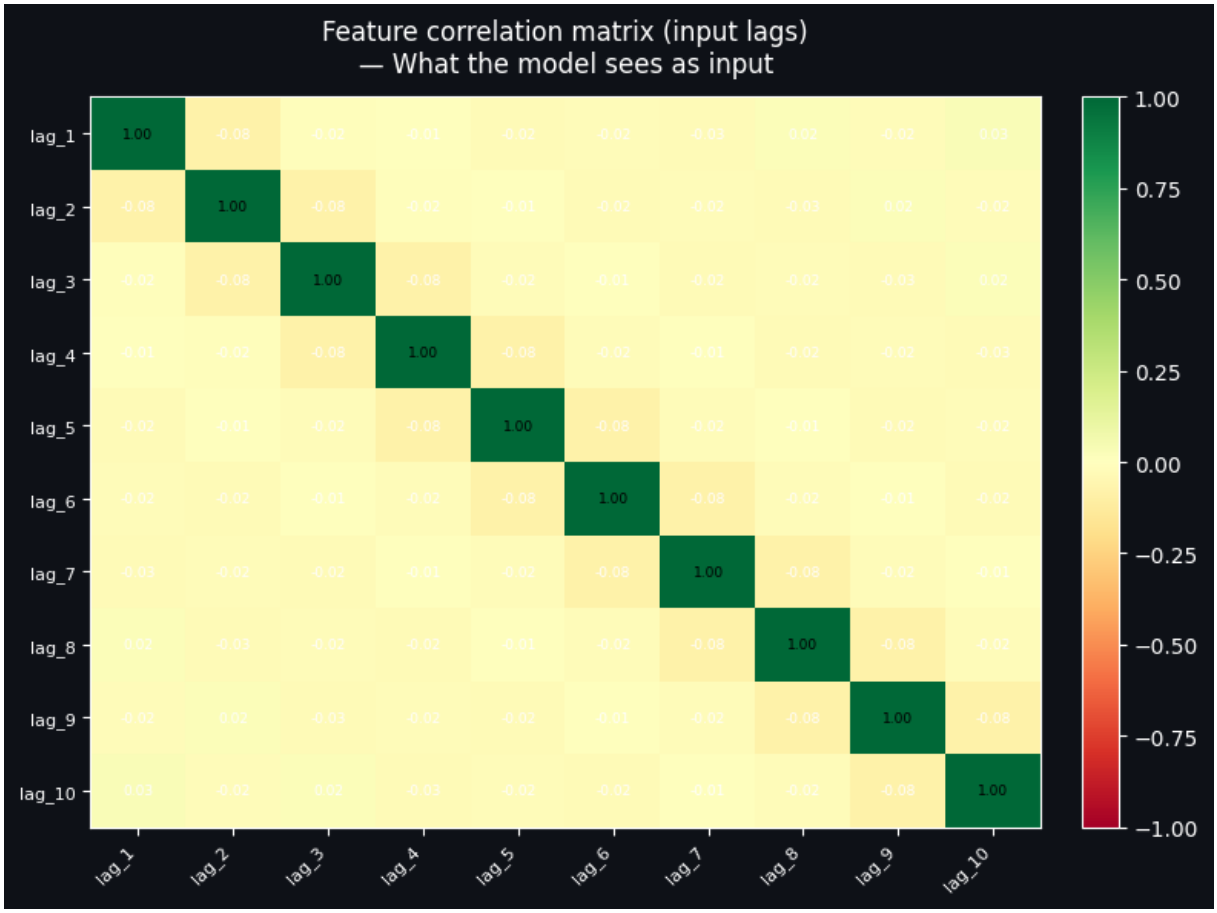

Figure 4: Pearson correlation matrix of the 10 lag features. Adjacent lags are positively correlated, reflecting the persistence of volatility regimes.

Adjacent lags are positively correlated due to volatility clustering: a large move on day $t-1$ is likely to be followed by a large move on day $t-2$. This introduces mild multicollinearity for linear models and gives sequential models (RNN, LSTM) a structural advantage.

2.7 Summary

Three statistical properties of the data guide the modelling choices throughout this report:

1. **Raw returns are unpredictable** (near-zero ACF) — the target must be the magnitude, not the sign, of returns.
2. **Volatility is autocorrelated** (ARCH effect) — past lags carry genuine predictive information.
3. **Two regimes coexist** — calm and crisis periods have fundamentally different statistical properties; a good model must handle both.

3 Linear Regression Baseline

This section establishes two classical baselines: ordinary least squares (OLS) regression and GARCH(1,1). Both are transparent and well-understood, making their failure modes informative for motivating the neural network architectures that follow.

3.1 Standard Linear Regression

3.1.1 Setup

An OLS model is trained on the 10 signed lag features with a standard train/test split. Features and target are scaled with a `StandardScaler` fitted on the training set only. The target $y_t = |r_t|$ is the realized absolute return, used throughout as a proxy for next-day volatility. The model takes the form

$$\hat{y}_t = \beta_0 + \sum_{k=1}^{10} \beta_k \, r_{t-k}.$$

3.1.2 Results

Table 3: Linear regression performance on the test set (scaled units).

Model	MAE	RMSE	QLIKE
Linear Regression (raw lags)	0.638	0.933	16.79
Linear Regression (engineered)	0.598	0.874	7.46

3.1.3 Coefficient Analysis

Table 4: OLS regression coefficients ranked by absolute magnitude.

Feature	Coefficient	$\lvert\beta_k\rvert$ rank
lag_9	-0.180	1
lag_8	-0.164	2
lag_10	-0.159	3
lag_7	-0.158	4
lag_5	-0.123	5
lag_6	-0.110	6
lag_3	-0.106	7
lag_1	-0.094	8
lag_4	-0.084	9
lag_2	-0.060	10

All coefficients are *negative* and small. This reveals the first fundamental limitation:

> **Symmetry blindness.** Linear regression on signed returns cannot treat $+x$ and $-x$ identically. Predicting volatility requires $f(x) \approx f(-x)$ (the absolute value function), which a linear model cannot represent. The model "gives up"

by assigning small negative coefficients and predicting near-zero values for all inputs.

3.1.4 Predicted vs. Actual

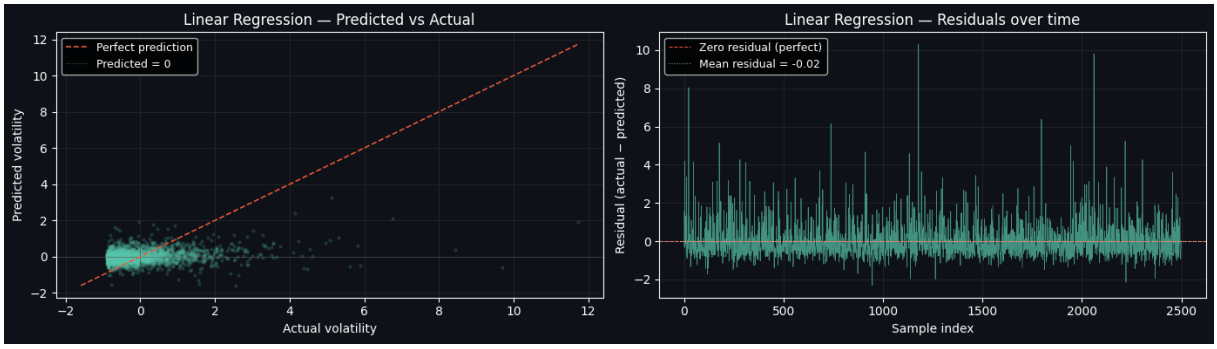

Figure 5: Linear regression: predicted vs. actual realized absolute return. The model systematically underpredicts during crisis episodes.

The model produces a near-constant prediction close to zero throughout the entire sample. Residuals are always positive and are dramatically amplified during crisis episodes.

3.2 Feature Engineering

Squaring the lag features forces the symmetry the model cannot learn from signed returns: $r_{t-k}^2 = |r_{t-k}|^2$ treats positive and negative shocks of equal magnitude identically. A *recency ratio* feature $r_{t-1}/\text{mean}(r_{t-2:t-5})$ is also added to capture short-term momentum in volatility.

Table 5: Impact of feature engineering on linear regression performance.

Features	MAE	RMSE	QLIKE
Raw signed lags	0.638	0.933	16.79
Squared lags + recency ratio	0.598	0.874	7.46
Improvement (Δ)	+6.3%	+6.4%	+55.6%

The QLIKE loss drops by 56%, showing that manual feature engineering substantially reduces systematic under-prediction. This foreshadows what the FNN can learn *automatically* from raw lags.

3.3 Regime Analysis

Table 6: Linear regression performance by market regime (raw lag features).

Regime	MAE	RMSE	QLIKE
Full	0.638	0.933	16.79
Calm	0.488	0.587	15.49
Crisis	1.987	2.367	28.46
Crisis / Calm ratio	4.1×	4.0×	1.8×

During crisis periods, MAE is **4× higher** than during calm periods. The model systematically underpredicts every volatility spike because it has no mechanism to detect a regime shift. This is the second structural limitation:

> **Regime blindness.** A single linear model fitted on the full sample cannot adapt its behaviour to different market conditions.

3.4 GARCH(1,1) Baseline

The GARCH(1,1) model provides a parametric volatility benchmark. The conditional variance follows:

$$\sigma_t^2 = \omega + \alpha\,\epsilon_{t-1}^2 + \beta\,\sigma_{t-1}^2,$$

where $\epsilon_t = r_t/\sigma_t$ are standardised residuals.

3.4.1 Estimated Parameters

Table 7: GARCH(1,1) maximum-likelihood parameter estimates.

Parameter	Estimate	Interpretation
ω (baseline variance)	0.0159	Long-run variance floor
α (shock sensitivity)	0.094	Reaction to new shocks
β (persistence)	0.895	Memory of past variance
$\alpha + \beta$ (total persistence)	0.989	Near unit-root variance process
Half-life of shock decay	60 days	Volatility shocks dissipate slowly

The near-unit-root persistence ($\alpha + \beta = 0.989$) confirms that volatility shocks take approximately **60 days** to decay to half their peak level.

3.4.2 Regime Performance

Table 8: GARCH(1,1) performance by market regime (unscaled returns).

Regime	MAE	RMSE	QLIKE
Full	0.00583	$0.000069^{\dagger}$	-3.98
Calm	0.00506	$0.000043^{\dagger}$	-4.17
Crisis	0.01281	$0.000296^{\dagger}$	-2.27
Crisis / Calm ratio	2.5×	6.9×	—

† These are MSE values; RMSE= $\sqrt{\text{MSE}}$.

GARCH improves on linear regression during calm periods but exhibits the same pattern of regime degradation: crisis MAE is 2.5× higher than calm MAE. Despite its elegant theoretical motivation, GARCH suffers from the third structural limitation:

> **Fixed functional form.** GARCH assumes a specific parametric update rule for variance. It cannot detect structural breaks or non-linear regime transitions.

3.5 Summary of Limitations

Table 9: Comparison of linear baseline models (MAE, full sample).

Model	MAE (full)	MAE (crisis)
Linear Regression (raw lags)	0.638	1.987
Linear Regression (engineered)	0.598	—
GARCH(1,1)	0.00583	0.01281

Three structural limitations motivate the turn to neural networks:

1. **Symmetry blindness** (linear regression on signed inputs cannot represent $|x|$).
2. **Regime blindness** (no model adapts its behaviour during crises).
3. **Fixed functional form** (GARCH assumes a pre-specified update rule).

A feedforward neural network with non-linear activations directly addresses limitation 1, and deeper architectures are expected to partially address limitations 2 and 3.

4 Feedforward Neural Network

4.1 Model and Validation

Architecture

The baseline FNN maps the L most recent signed log-returns to a scalar one-day-ahead volatility forecast through two hidden layers:

$$\mathbf{x} \in \mathbb{R}^{L} \xrightarrow{\text{Dense}_{32}} \mathbb{R}^{32} \xrightarrow{\text{Dense}_{16}} \mathbb{R}^{16} \xrightarrow{\text{Dense}_{1}} \hat{y} \in \mathbb{R}_{+}.$$

Both hidden layers use **ReLU** activations and **He normal** initialisation; the output neuron is linear. This choice of activation directly addresses the *symmetry blindness* of the linear baseline: two composed ReLU layers can approximate $|x|$ from signed inputs, recovering the signal in r_t without manual feature engineering.

Implementation validation

The custom NumPy framework (`dl_utils`) is cross-checked against an equivalent PyTorch implementation before any tuning.

Table 10: Custom NumPy implementation vs. PyTorch reference and linear baseline (lag = 10).

Implementation	MAE	Note
Custom `dl_utils` (NumPy)	0.6046	dropout = 0.5 on layer 1
PyTorch reference	0.5850	no dropout
Linear regression baseline	0.6377	

Both neural implementations beat the linear baseline. The small gap between them stems from the dropout regularisation applied only in the NumPy model: dropout slows convergence, raising the early-stopped MAE slightly.

4.2 Hyperparameter Tuning

All experiments in this section share the same default training protocol unless stated otherwise: MSE loss, batch size 32, maximum 300 epochs, early stopping with patience = 10 evaluated every 5 epochs.

4.2.1 Activation Function

Four activations are compared on the 32→16→1 architecture with plain SGD ($\alpha = 10^{-3}$).

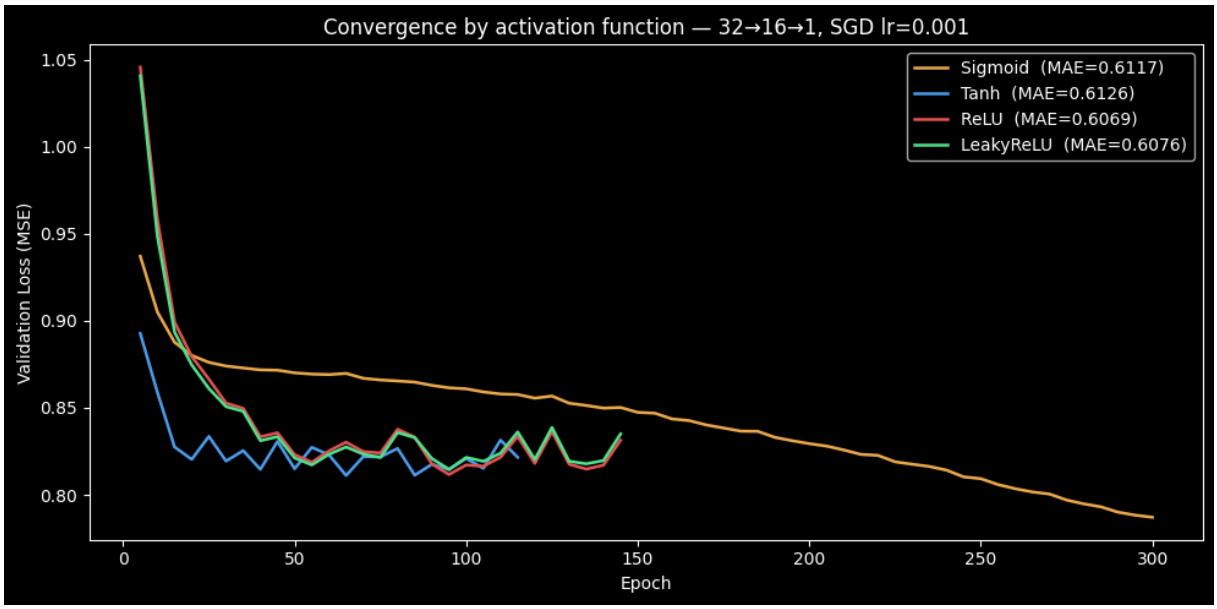

Figure 6: Validation loss (MSE) over 300 epochs for four activations. ReLU (red, MAE = 0.607) reaches the best test MAE; Sigmoid (orange) is still declining at epoch 300.

- **Sigmoid** (MAE 0.612): gradient saturation dramatically slows learning; the validation loss has not plateaued by epoch 300. The model does ultimately converge at this shallow depth after `StandardScaler` normalisation, but requires far more epochs than the others.
- **Tanh** (MAE 0.613): converges the fastest (before epoch 30), yet it reaches the *worst* test MAE. Fast training convergence does not imply good generalisation.
- **ReLU** (MAE 0.607, best): rapid, stable convergence with mild oscillations attributable to occasional dead-neuron events.
- **LeakyReLU** (MAE 0.608): virtually identical to ReLU. After `StandardScaler` normalisation the input distribution is centred, so the dying-neuron advantage of LeakyReLU is negligible.

Decision: ReLU for all subsequent experiments.

4.2.2 Weight Initialisation

Five initialisation schemes are compared, all paired with ReLU.

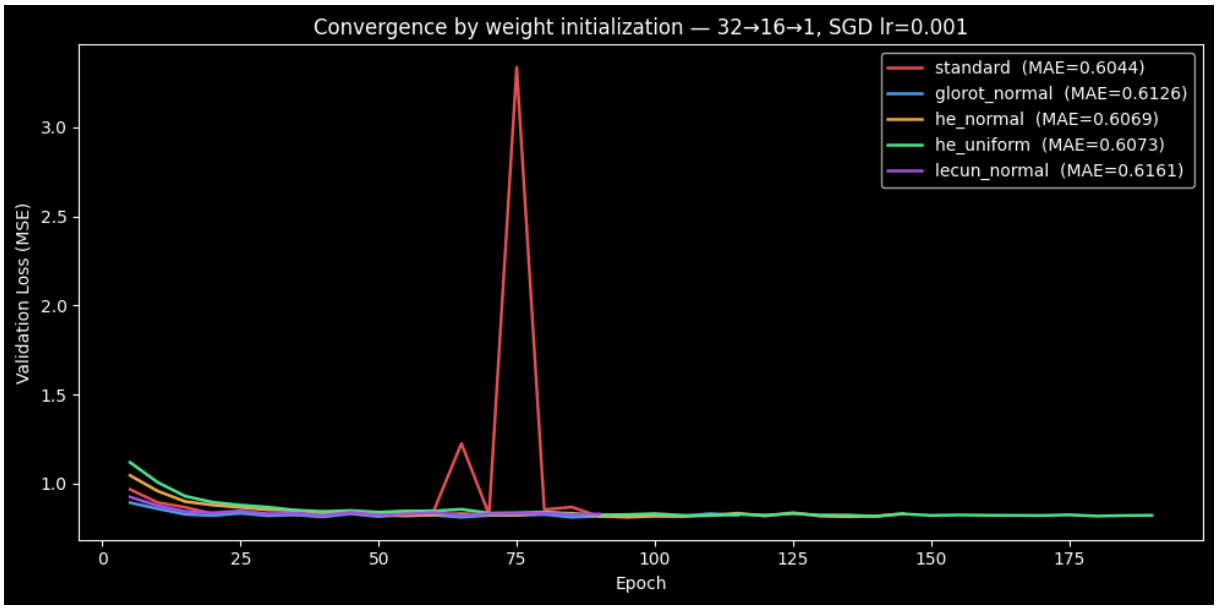

Figure 7: Convergence by initialisation scheme. Standard $\mathcal{N}(0,1)$ (red) triggers a gradient explosion at epoch ≈ 75 (MSE peaks at 3.3) but ultimately converges to the lowest test MAE. He normal/uniform converge stably and reach comparable performance.

Table 11: Final test MAE by initialisation scheme (ReLU, SGD $\alpha = 10^{-3}$).

Initialisation	MAE
Standard $\mathcal{N}(0,1)$	0.604
He normal	0.607
He uniform	0.607
Glorot normal	0.613
LeCun normal	0.616

Standard initialisation achieves the *lowest* final MAE but only through an unreliable mechanism: the gradient explosion near epoch 75 (MSE $\rightarrow$ 3.3) accidentally perturbs the weights out of a shallow basin. On a harder problem or a deeper network this spike would cause permanent divergence. He normal reaches near-identical performance (+0.003) with *monotone, predictable* convergence, and is theoretically motivated: for ReLU layers, $\mathrm{Var}(W) = 2/n_{\mathrm{in}}$ ensures that activation variance is preserved layer-to-layer, preventing both vanishing and exploding signals.

Decision: He normal for all subsequent experiments.

4.2.3 Optimizer and Momentum

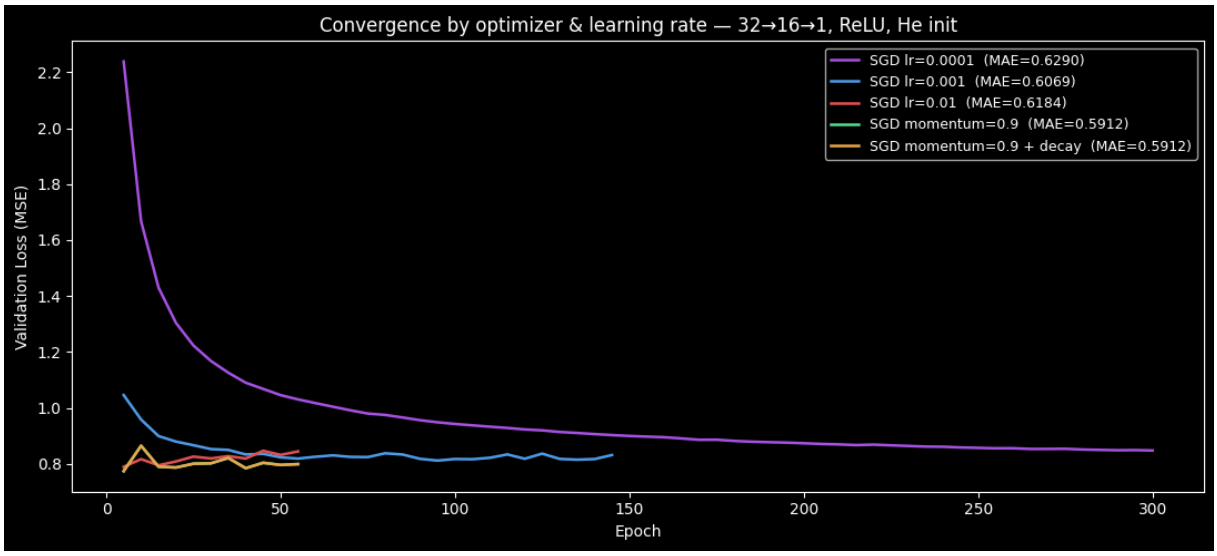

Figure 8: Convergence by optimizer. Plain SGD at three learning rates and SGD with momentum $\mu = 0.9$ (with and without LR decay). Momentum dominates all plain-SGD variants; adding decay changes nothing under early stopping.

Learning rate and effect of momentum.

- $\alpha = 10^{-4}$: convergence is too slow (MAE 0.629 at epoch 300, still declining).
- $\alpha = 10^{-3}$: stable convergence (MAE 0.607).
- $\alpha = 10^{-2}$: early overshoot followed by convergence to a worse minimum (MAE 0.618).
- SGD + momentum $\mu = 0.9$ (MAE 0.591): reaches the final MAE of plain SGD ($\alpha = 10^{-3}$) roughly $10\times$ faster, then converges further.
- SGD + momentum + LR decay (MAE 0.591): identical to plain momentum. Early stopping triggers before the decay schedule has a meaningful effect, rendering it redundant.

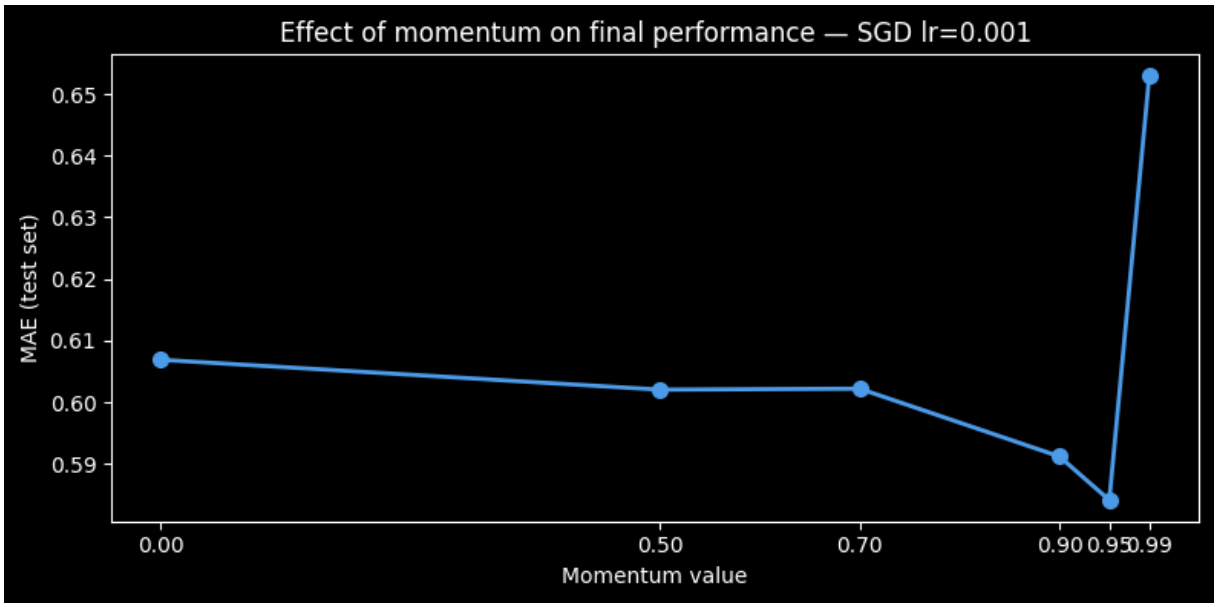

Figure 9: Test MAE as a function of momentum coefficient μ. Performance improves steadily to $\mu = 0.95$; at $\mu = 0.99$ the model diverges.

Table 12: MAE by momentum coefficient (He normal, ReLU, lag = 10).

μ	MAE
0.00	0.607
0.50	0.602
0.70	0.602
0.90	0.591
0.95	**0.587**
0.99	diverges

Momentum sweep. Performance improves steadily up to $\mu = 0.95$, then diverges at $\mu = 0.99$. At very high momentum the accumulated velocity overpowers the corrective gradient signal, causing the weights to oscillate with growing amplitude.
Decision: $\mu = 0.95$ for all subsequent experiments.

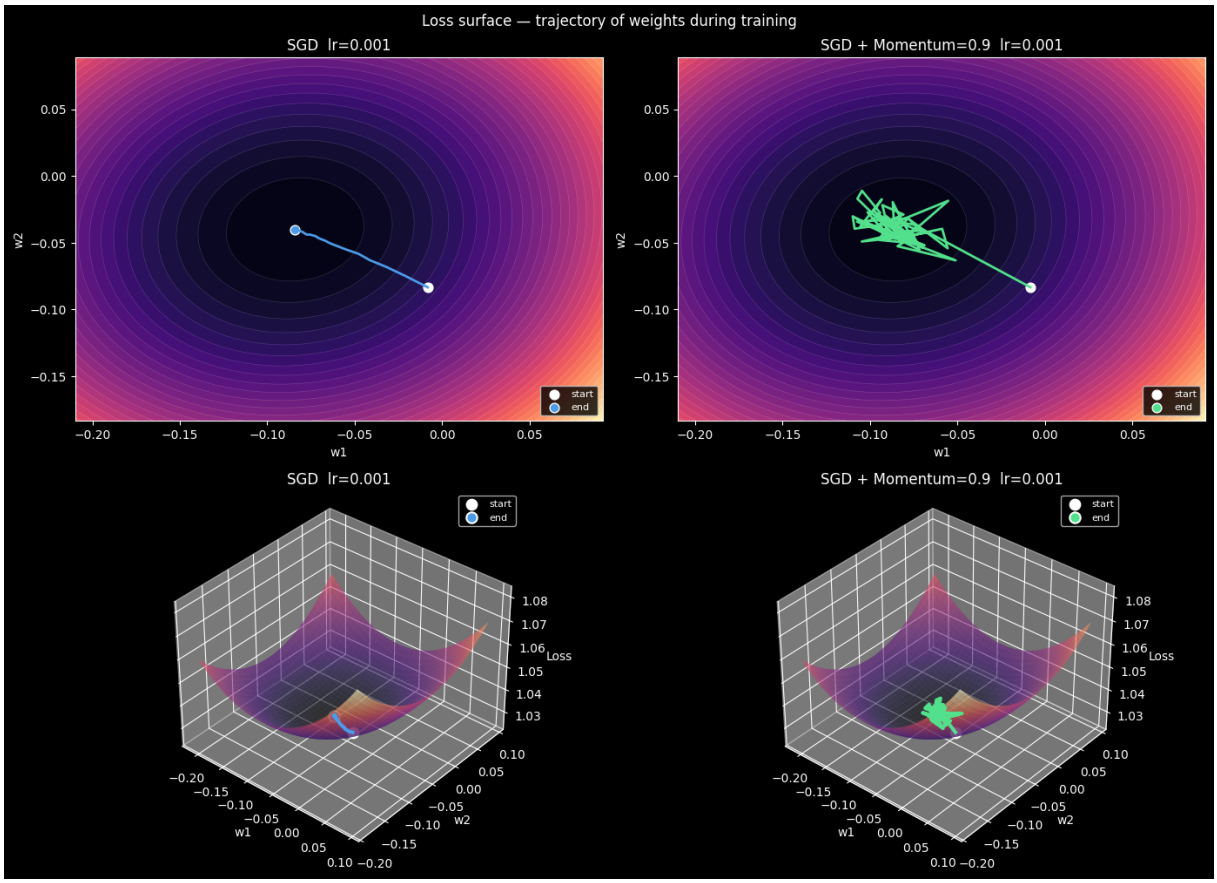

Figure 10: 2-D loss-surface contours (top) and 3-D surfaces (bottom) with weight trajectories for plain SGD (left, blue) and SGD+momentum $\mu = 0.9$ (right, green). Momentum follows a wider arc but descends to a darker (lower-loss) region.

Loss surface geometry. Plain SGD takes short, monotone gradient steps and stalls in the first basin it enters. Momentum accumulates velocity in the consistent gradient direction, allowing it to traverse flat saddle regions and settle in a deeper minimum — visible in the contour plot as the green trajectory reaching a darker zone unreachable by the blue one.

4.2.4 Temporal Window

The lag L is swept across $\{1, 3, 5, 10, 20, 30, 45, 60\}$ using the best configuration found above (ReLU, He normal, SGD $\mu = 0.95$).

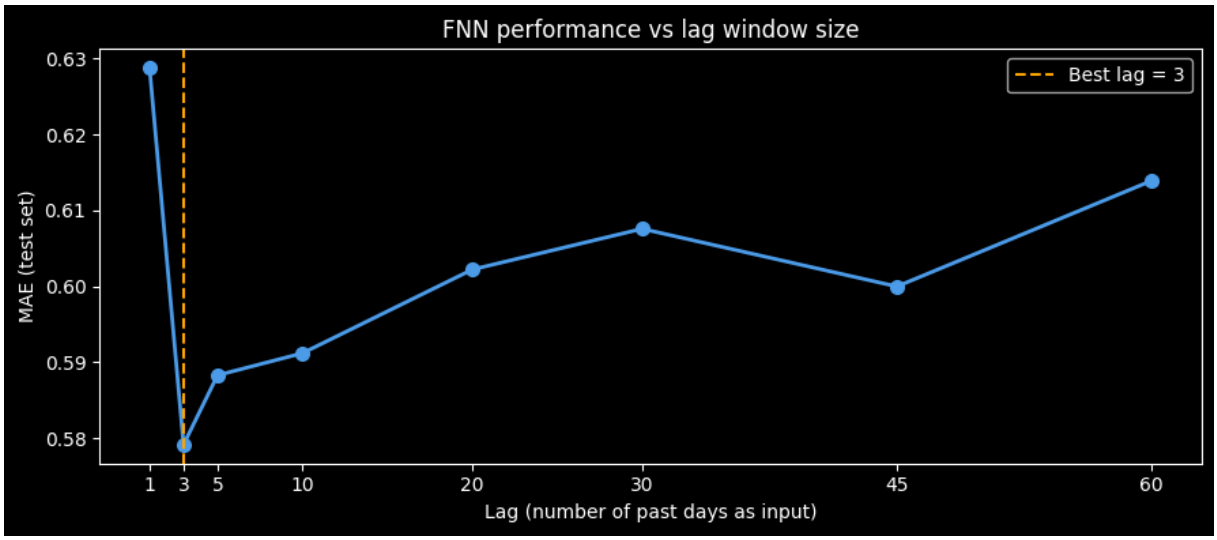

Figure 11: Test MAE as a function of lag window L. Performance peaks at $L = 3$ (MAE 0.578, dashed orange); it generally increases for larger windows with a local dip at lag 45.

Table 13: Test MAE by lag window (ReLU, He normal, SGD $\mu = 0.95$).

Lag L	MAE
1	0.629
3	**0.578**
5	0.589
10	0.591
20	0.602
30	0.608
45	0.600
60	0.614

Performance peaks at $L = 3$ and generally degrades beyond, though not strictly monotonically (lag = 45 recovers slightly below lag = 30, a seed-dependent fluctuation). The pattern reflects a structural limitation of the FNN: all lag features receive independently learned fixed weights, so the network has no mechanism to discount distant, noisy lags. Beyond 3 days, the additional information added by older returns is outweighed by the noise they introduce.
Decision: $L = 3$ for all subsequent experiments.

4.3 Diagnostics: Inside the Trained Network

The figures in this section do not drive hyperparameter choices; they explain *why* the chosen configuration (ReLU + He normal) works and why the bad configuration (Sigmoid + standard init) fails.

4.3.1 Weight and Activation Distributions

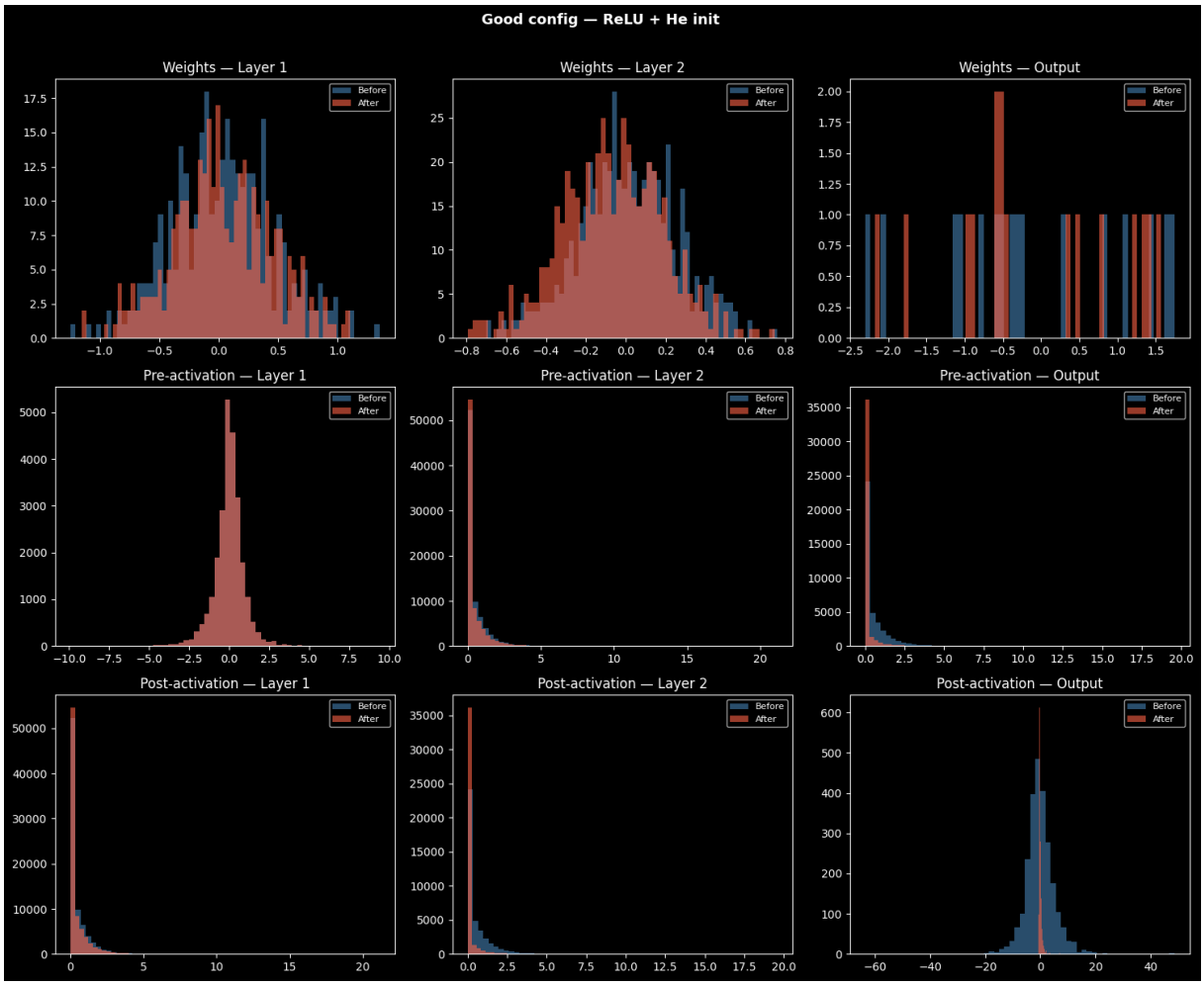

Figure 12: *Good* configuration (ReLU + He normal). Rows: weight distributions, pre-activations, post-activations. Columns: layers 1, 2, output. Blue = before training; red = after. All three layers show substantial weight spreading and healthy activation ranges.

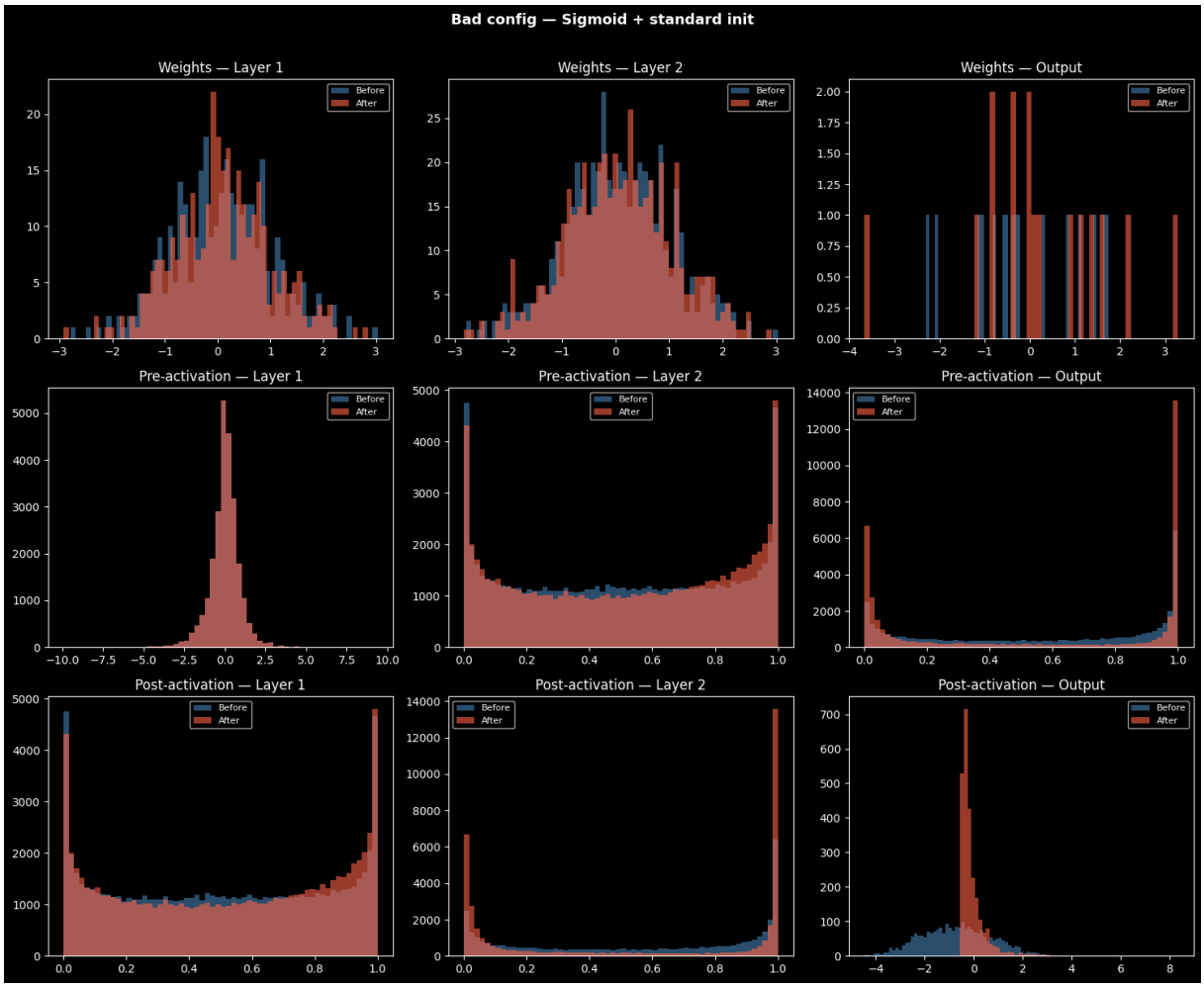

Figure 13: *Bad* configuration (Sigmoid + standard init). Same layout. Weights barely move during training; layer 2 post-activations collapse to the narrow interval [0.9, 1.0] — the saturation zone — transmitting essentially no information to the output.

With ReLU + He, weight distributions broaden during training and pre-activations span a wide range: every neuron is active and contributing. With Sigmoid + standard init, the gradient explosion compresses post-activations into Sigmoid's saturation zone; backpropagated gradients become near-zero and learning stalls. The distribution collapse renders vanishing gradients directly visible without gradient inspection.

4.3.2 Gradient Flow

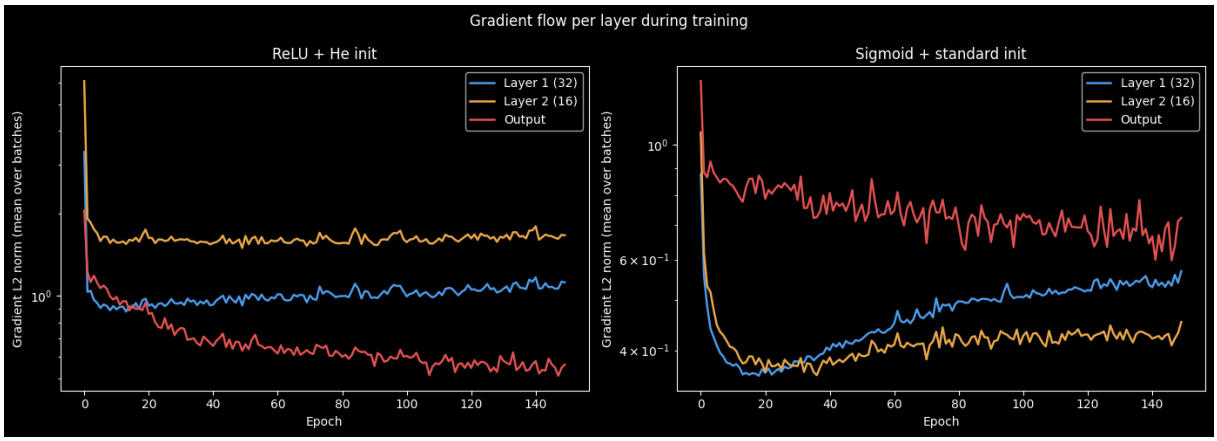

Figure 14: Gradient ℓ_2-norms per layer over 150 epochs. Left (ReLU + He): all layers maintain comparable norms throughout. Right (Sigmoid + standard): early layers receive gradients 2–3× smaller than the output layer.

With ReLU, the derivative equals 1 for every active neuron, so gradients propagate backward without attenuation: all layers update at the same scale. With Sigmoid, the derivative is bounded by 0.25 and approaches 0 in saturation; compounded across layers this concentrates learning at the output while the input layer stagnates. In a deeper network this effect would be catastrophic.

4.3.3 Weight Evolution During Training

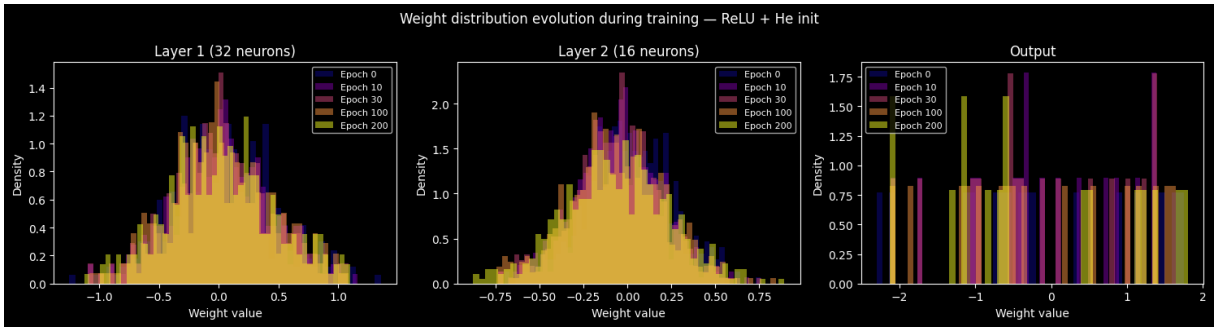

Figure 15: Weight distribution snapshots at epochs 0, 10, 30, 100, 200 (ReLU + He normal, best config). All three layers exhibit progressive spreading from a tight initialisation to broad, learned distributions.

At epoch 0 the He initialisation produces narrow Gaussian-like distributions in all layers. By epoch 200, layers 1 and 2 have spread considerably and the output layer has developed a multimodal structure. The monotone spreading across every layer confirms that no part of the network is frozen or dead: gradient updates reach and reshape all weights throughout training.

4.4 Structural Limitations

Despite a 9 % improvement over the linear baseline, the FNN has two architectural limitations that no amount of hyperparameter tuning can address.

4.4.1 Order Blindness

The FNN receives the L lag features as a flat, unordered vector. To measure how much temporal ordering it *implicitly* exploits, we evaluate the trained model on 30 random permutations of the input features and report the mean MAE gap.

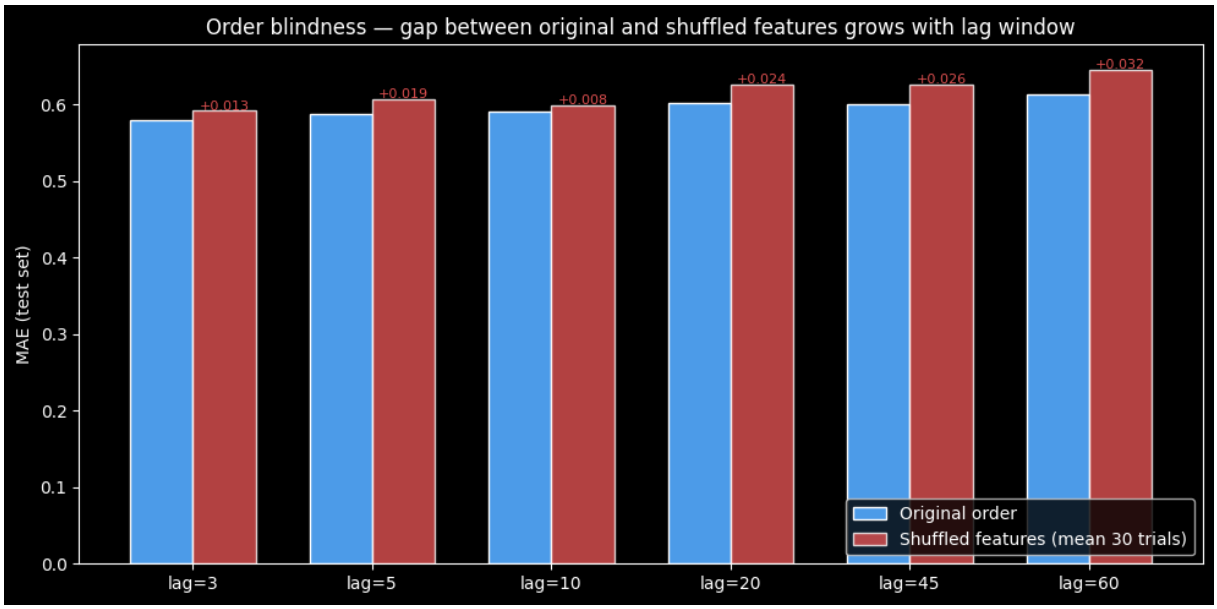

Figure 16: Test MAE for original feature order (blue) vs. randomly shuffled features (red, mean of 30 trials) at each lag window. The degradation gap grows from +0.013 at lag 3 to +0.032 at lag 60.

Table 14: Original vs. shuffled-input MAE by lag window.

Lag L	Original	Shuffled	Gap
3	0.578	0.591	+0.013
5	0.589	0.608	+0.019
10	0.591	0.599	+0.008
20	0.602	0.626	+0.024
45	0.600	0.626	+0.026
60	0.614	0.646	+0.032

The network has learned to exploit recency through its weight magnitudes — recent lags receive larger learned weights — but this is an emergent property of optimisation, not a guarantee of the architecture. The growing gap with lag window confirms that richer temporal structure is present and the FNN partially uses it; a recurrent architecture would process inputs sequentially, making temporal ordering a structural rather than a learned property.

4.4.2 Regime Blindness

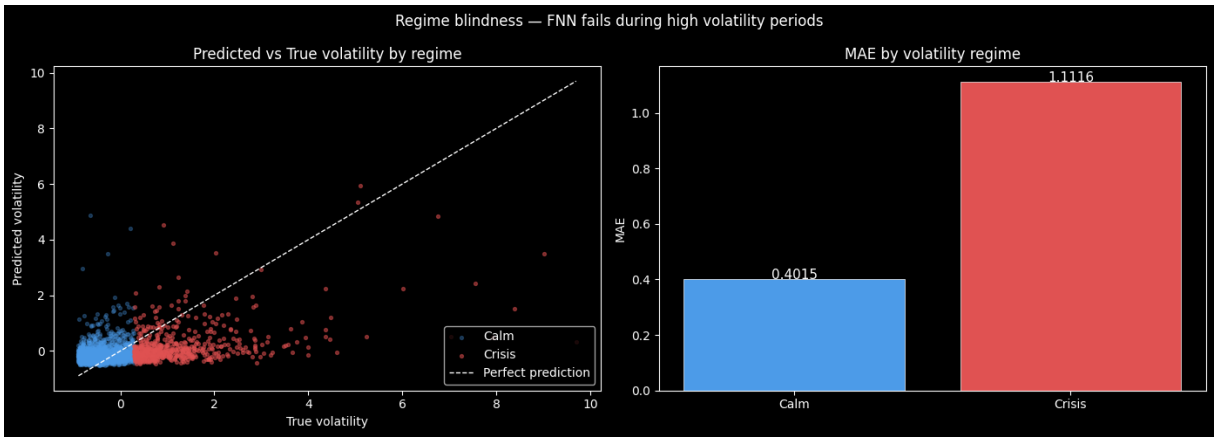

Figure 17: Left: predicted vs. true volatility coloured by regime (calm = blue, crisis = red). Crisis predictions cluster near zero regardless of the true spike magnitude. Right: MAE by regime — crisis MAE is 2.77× higher than calm MAE.

Table 15: FNN MAE by volatility regime (75th-percentile threshold, $L = 3$).

Regime	MAE	Share
Calm ($\sigma_t < \sigma^*_{75}$)	0.402	75 %
Crisis ($\sigma_t \geq \sigma^*_{75}$)	1.112	25 %
Crisis / Calm ratio	2.77×	

The FNN makes each prediction from a static 3-day snapshot and then resets its state entirely. It cannot accumulate the context of a sustained volatility cluster building over days or weeks. Without persistent memory, *"high volatility today ⇒ high volatility tomorrow"* is not learnable.

4.5 Summary

Table 16: Best FNN configuration and rationale.

Parameter	Choice	Rationale
Activation	ReLU	Best test MAE; stable gradient propagation
Initialisation	He normal	Stable convergence; variance-preserving for ReLU
Optimizer	SGD $\mu = 0.95$	Fastest convergence; best generalisation
Lag window	$L = 3$	Beyond 3 days, distant lags add noise

With this configuration the FNN achieves **MAE** = 0.579 on the test set versus **0.638** for the linear baseline — a **9 % improvement** driven entirely by the non-linear activation resolving symmetry blindness.

Two structural limitations, demonstrated empirically, motivate the architectures developed in the following sections:

- **Order blindness**: the FNN has no positional encoding; a recurrent architecture is needed to make temporal ordering a structural property rather than an optimisation artefact.
- **Regime blindness**: without a persistent hidden state, the model cannot exploit volatility clustering; an RNN or LSTM is needed to carry context across time steps.

5 Recurrent Neural Network

5.1 Architecture

5.2 Training Setup

5.3 Results

5.4 Limitations

6 Long Short-Term Memory

6.1 Architecture

6.2 Training Setup

6.3 Results

6.4 Cross-Model Comparison

6.5 Conclusion